



HOMEWORK 2 REPORT

Mars Rover: Red Planet Rescue - Reinforcement Learning

UMAIR AHMAD

S031654

Department of Computer Science

CS 451 : Introduction to Artificial Intelligence — Spring 2026

Abstract

This report summarizes the implementation of the Mars Rover reinforcement learning homework. The work contains a model-based Value Iteration planner, parameter choices for the Canyon Grid policy profiles, and a model-free tabular Q-learning agent with epsilon-greedy exploration. Value Iteration was implemented using batch Bellman backups so that each iteration computes the next value table from the previous value table. Q-learning was implemented using a Counter-backed Q-table with default zero values for unseen state-action pairs, random tie-breaking among equally good greedy actions, epsilon-greedy action selection, and the standard temporal-difference update. The final implementation was tested with the supplied self-check script and the public autograder. The self-check passed all 16 checks, and the public autograder returned 75 out of 75 points across Q1 through Q5.

Keywords: reinforcement learning, value iteration, Q-learning, epsilon-greedy exploration, Mars Grid

1 Problem Overview

The homework asks the rover to act optimally in stochastic grid worlds. In the planning part, the transition model is known, so the rover can compute an optimal value function before acting. In the learning part, the rover learns directly from sampled transitions without using the transition model. The submitted code only modifies the three student files required by the assignment: `value_iteration_agents.py`, `qlearning_agents.py`, and `analysis.py`.

2 Value Iteration

For Q1, the planner estimates the optimal state value function with the Bellman optimality equation:

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]. \quad (1)$$

The important implementation point is that the update is batch-based. A fresh `Counter` is created at every sweep, all new values are computed from the previous `self.values`, and `self.values` is replaced only after the full sweep finishes. Terminal states and states with no legal actions are left at value zero.

The action-value helper computes:

$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]. \quad (2)$$

The policy helper then returns the legal action with the largest computed Q-value, or `None` for terminal states.

3 Canyon Grid Policy Profiles

For Q2, each function in `analysis.py` returns a tuple $(\gamma, \text{noise}, \text{living reward})$. The chosen parameters follow the intended behavior described in the homework: low discount makes the rover myopic, zero noise makes cliff-hugging safer, nonzero noise makes cliff-adjacent routes dangerous, and positive living reward can make indefinite wandering preferable.

4 Q-Learning and Exploration

For Q3 through Q5, the Q-table is stored in `self.q_values`, a `Counter` keyed by $(\text{state}, \text{action})$. Missing entries return zero, so unvisited actions are still considered during maximization. The learned value of a state is:

$$V(s) = \max_a Q(s, a). \quad (3)$$

Table 1: Q2 parameter choices.

Function	Intended profile	Discount	Noise	Living reward
question2a	close exit, risky route	0.2	0.0	-1.0
question2b	close exit, safe route	0.2	0.2	-1.0
question2c	distant depot, risky route	0.9	0.0	-0.1
question2d	distant depot, safe route	0.9	0.2	-0.1
question2e	avoid termination	0.9	0.2	1.0

If there are no legal actions, the value is defined as zero.

The greedy action method collects all actions whose Q-value equals the best value and returns one of them uniformly at random. This matters because at initialization all Q-values are tied at zero; deterministic tie-breaking would bias the learner toward one fixed action.

The learning update is the standard one-step temporal-difference rule:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') \right]. \quad (4)$$

The epsilon-greedy action method explores with probability ϵ by selecting a random legal action, and otherwise exploits by choosing the current greedy action. Exploration is allowed to select the same action as the greedy policy, which is the expected epsilon-greedy behavior.

5 Implementation Summary

The final implementation makes the following changes:

- `value_iteration_agents.py`: implemented batch value iteration, model-based Q-value computation, and greedy policy extraction.
- `analysis.py`: filled all five Canyon Grid parameter functions.
- `qlearning_agents.py`: implemented Q-value lookup, value extraction, random tie-broken greedy action selection, epsilon-greedy action selection, and the Q-learning update rule.

No project infrastructure files, environment files, grading files, layouts, or test cases were modified.

6 Validation Results

The implementation was validated with both supplied commands:

- `python check.py`: 16 passed, 0 failed.
- `python autograder.py --no-graphics`: 75/75 total points.

The autograder breakdown was Q1: 20/20, Q2: 20/20, Q3: 20/20, Q4: 8/8, and Q5: 7/7. This confirms that the planner, analysis parameters, Q-learning update, epsilon-greedy exploration, Mars deployment test, and crawler smoke test all passed the supplied public tests.

7 Conclusion

The completed homework implements both model-based planning and model-free learning for the Mars Rover domain. Value Iteration uses correct batch Bellman updates, while Q-learning uses a general environment-independent implementation that also works for the crawler domain. The final submission is restricted to the required files plus this PDF report.